

Does the PCPG update implement a natural gradient?

Measurement report · branch fixing-pcpg · probes run on CPU, jax 0.4.38 / jpc 1.0.0 / equinox 0.13.8 / diffraX 0.7.2

Answer: no. Inference does not move the PC update toward the natural gradient — at equilibrium it moves it away. The only natural-gradient content present comes from the target construction (the `natural_target` flag), not from predictive coding. Separately, the adaptive trust-region rescaling that PC is expected to supply is numerically inert in this architecture: it changes step length, not direction.

Two findings qualify that. First, production inference was running at ~1% of the integration needed to settle, so until it was fixed the equilibrium claims were being tested on activities that had barely moved. Second, once settling was fixed and actually ablated, it did change learning on the bandit — so the “inference is inert” result is specific to the geometry it was measured on and does not generalise.

1. What is actually being compared

The probe freezes one policy and one batch, then computes the PC weight update and several reference directions on that same frozen state. Everything is a direction in parameter space, so the comparisons are cosines. No training is involved.

direction	definition	the hypothesis it stands for
$\Delta(t1)$	$-\partial E / \partial \theta$ at settled activities	what PCPG actually does
d_SGD	$-\nabla L$	vanilla policy gradient
d_OUT	$-J^T P u$, $P = [\sigma^2, \frac{1}{2}]$	output metric only — what the natural target encodes
$d_NG(\lambda)$	$-(F + \lambda I)^{-1} \nabla L$, $F = J^T F_out J$	the natural gradient (both factors)
d_BP	$-\nabla \theta E$ (backprop on the energy)	“PC $\approx$ backprop”
d_EQ	$-\nabla \theta [\frac{1}{2} r^T S^{-1} r]$, $S = I + \Sigma B B^T$	the equilibrium / trust-region prediction

F is the exact Fisher of the policy, applied matrix-free by jvp/vjp plus CG, so there is no action-sampling noise; λ is swept and the best-fit value is part of the measurement. $F_out = \text{diag}(1/\sigma^2, 2)$ is the Gaussian output Fisher. Alongside the ordinary cosine we report the **Fisher-metric cosine** $\cos_F$, which is parameterisation-invariant and equals 1 exactly when a direction buys the most loss improvement per unit KL.

Setup and the harness check

Policy is `jpc.make_mlp 17 → 64 → 12 (tanh)`, batch $n = 256$ drawn on-policy, advantages standardised, seed 0. Four regimes place the σ head, and two target families are run in each: Euclidean (mean offset $ts \cdot A \cdot (z - \mu) / \sigma^2$, the production default) and natural ($ts \cdot A \cdot (z - \mu)$, the $1/\sigma^2$ amplifier removed). Inference time $t1$ is swept from 0 to 4096.

The harness has a built-in anchor: at $t1 = 0$ no inference has happened, so the update must be exactly final-layer-only and its final-layer block must equal d_BP . Both come out at 1.000000 to float precision in every cell, which is what licenses reading the rest.

2. Reproduction of the earlier pilot

Findings F1-F5 predate this work. They reproduce on an independent machine, so what follows builds on a checked base rather than a single run.

quantity	this run	pilot
floor / Euclidean $\cos(\Delta, d_{NG^*})$, $t1=20 \rightarrow eq$	0.616 $\rightarrow$ 0.124	0.68 $\rightarrow$ 0.11
floor / Euclidean $\cos_F(\Delta, d_{NG^*})$, $t1=20 \rightarrow eq$	0.904 $\rightarrow$ 0.281	0.93 $\rightarrow$ 0.27
mixed / Euclidean $\cos(\Delta, d_{NG^*})$ at equilibrium	0.202	0.20
mixed / natural $\cos(\Delta, d_{NG^*})$ at equilibrium	0.618	0.60
harness anchors at $t1 = 0$	1.000000	1.000000

Read the arrows as $t1 = 20$ (the production operating point) $\rightarrow t1 = 4096$ (equilibrium). Alignment with the natural gradient falls as inference proceeds, in every cell. That is the core negative result: settling does not buy natural-gradient geometry, it spends it.

3. New: the fixed- σ control

A fourth regime was added as a control: zero the `log_std` rows of the final weight so $\sigma = \exp(\text{bias})$ no longer depends on the state, with $\text{bias} = 0$ giving $\sigma \equiv 1$. This is PPO's parameterisation. It answers “how much geometry is at stake at all?”

regime	σ range	$\cos(d_{SGD}, d_{NG})$	$\cos(d_{OUT}, d_{NG})$
fixed (control)	1.000 – 1.000	0.941	0.933
init	0.419 – 1.649	0.925	0.850
mixed	0.135 – 1.162	0.622	0.723
floor	0.135 – 0.297	0.341	0.500

With σ constant, $F_{out} = \text{diag}(1, 2)$ is nearly isotropic, so the natural gradient and the vanilla gradient nearly coincide (0.941). The claim “ $PC \approx NG$ ” is close to unfalsifiable there, because almost any direction is $\approx NG$. Heteroscedastic σ is what creates the gap the claim is about, and it is widest at the σ floor (0.341).

Consequence for how this gets written up: the fixed- σ regime is the right control and the wrong test case. It calibrates how much geometry is at stake; it is never evidence that the update is natural.

4. New: the `log_std` clamp binds through the target, not through σ

The earlier F5 attributed a loss of target fidelity to σ being pinned at its floor. The control shows that mechanism is at best secondary. In the fixed regime $\sigma \equiv 1$ and **0.0%** of raw `log_std` outputs are out of bounds, yet fidelity on the `log_std` half is still 0.6995 (Euclidean) / 0.7902 (natural), not 1.0.

The reason is that the clip applies to `log_scale + offset`, so a target leaves the 2.5-wide window whenever the offset is large — wherever σ sits. Measured at $\sigma \equiv 1$:

target_scale	<code>log_std</code> targets clipped	median offset
1.0 (probe default)	27.3%	0.43
10.0 (bench ts10)	80.9%	4.27

Every bench config uses `ts10`, so roughly four in five `log_std` targets are truncated in production — in the configuration that was supposed to be the clean one. Any statement about what the `log_std` channel encodes has to be conditioned on `target_scale`, not only on σ .

5. New: why the trust-region prediction yields no natural gradient

Two separate things are true here, and conflating them is easy.

5a. The theorem and “natural gradient” are different claims

$S = I + \sum_l B_l B_l^T$ with $B_l = \partial \text{out} / \partial z_l$ is built from the **network Jacobian** — a property of the architecture. The Fisher $F = J^T F_{\text{out}} J$ is built from the **output distribution's** KL geometry — a property of the policy. Different objects. Nothing inside PC ever sees the policy distribution; PC receives a regression target. So distributional geometry has to be injected at the output, which is exactly what the natural target does.

5b. And in this architecture S is inert

The existing probe reported $\cos(\Delta, d_{\text{EQ}})$ but never how far d_{EQ} is from plain backprop — which is the entire content of the trust-region claim. Measuring that directly:

regime	hidden layers	eig(S)	cond(S)	cos(d_BP, d_EQ)	cos(d_EQ, d_NG)
mixed	1 (bench)	1.04 – 1.6	1.5	0.9948	0.931
mixed	2	1.11 – 1.8	1.6	0.9936	0.921
mixed	4	1.18 – 1.9	1.6	0.9921	0.912
mixed	8	1.18 – 2.0	1.7	0.9853	0.897
fixed	1	1.00 – 1.5	1.5	0.9930	0.632
fixed	4	1.00 – 1.7	1.7	0.9903	0.278
fixed	8	1.00 – 1.8	1.8	0.9949	0.076

$\cos(d_{\text{BP}}, d_{\text{EQ}}) \approx 0.99$ everywhere: PC-at-equilibrium is directionally backprop. The cause is the condition number, not the magnitude — a preconditioner only rotates a gradient if it is anisotropic, and $\text{cond}(S) \approx 1.5$ is nearly isotropic, so S rescales step length and leaves direction alone.

Both obvious escapes are closed. **Depth does not rescue it** — the sum runs over hidden layers, so growth was expected, but cond creeps only $1.5 \rightarrow 1.8$ from 1 to 8 hidden layers. **It is not a tanh-linearisation artifact** — in a linear network, where the equilibrium result is exact rather than first-order, $\cos(d_{\text{BP}}, d_{\text{EQ}})$ is still 0.985–0.995.

An important limit on this instrument

Innocenti et al. (arXiv:2305.18188) state the headline prediction as PC escapes saddle points faster than backprop, with the dynamics interpolating between backprop's gradient direction and a trust-region direction. Two things follow. Backprop is one endpoint of that interpolation, so $\cos(\Delta, d_{\text{BP}}) \approx 1$ is the theory's own degenerate case rather than a refutation. And d_{EQ} goes inert at a saddle: B_l is a product of weight matrices, so it vanishes as the weights approach the origin saddle of a linear net. Scaling every weight matrix by ε in a depth-5 linear net:

ε	cond(S)	cos(d_BP, d_EQ)
1.00	1.62	0.99097
0.30	1.04	0.99995
0.10	1.00	1.00000
0.01	1.00	1.00000

So the rescaling is most inert exactly where the paper predicts the largest effect. d_{EQ} encodes a local direction-rescaling reading of the theorem; saddle escape is a claim about the landscape and the dynamics. These results should be read as “inference supplies no natural-gradient or output-metric content”, which is what they measure, and **not** as evidence against the trust-region result. Testing that needs a saddle-escape experiment under plain gradient descent, which does not exist yet. (The paper's full text was not retrievable from the run environment, so this rests on its title and abstract.)

6. The finding that undercut all of the above: inference was never settling

jpc defines the PC energy as a batch mean, $F = (1/2N) \sum_i \|\cdot\|^2$. Each sample's activities appear only in their own term, so $\partial F / \partial z_i$ carries a factor $1/N$ and every activity's velocity is divided by N . Because dividing a gradient flow by a positive constant cannot move its fixed point, this is purely a change of clock rate — but it means $\max_t1$ is an ODE end time whose effective per-sample time is $\tau = t1/N$.

Measured: the residual is a function of τ **alone** ($N = 64$ and $N = 256$ agree to three decimals), so one curve covers every batch size.

$\tau = t1/N$	residual / initial
0.00977	0.988 ← production ($\max_t1=20$, $N=2048$): 1.2% settled
0.3	0.645
1.0	0.218
3.0	0.026 90% settled
10.0	0.002 ← converged; $\tau=30$ and $\tau=100$ do not improve

Production therefore removed 1.2% of the residual where $\tau \approx 10$ was needed — a $\sim 1000\times$ shortfall. Raising $\max_t1$ cannot fix it: jpc hard-codes a stop at $t = 4096$, which is $\tau = 2$ (93.8% settled) at bench N , while convergence needs $t1 \approx 20,480$. A second bug compounds it — jpc's steady-state early-stop tests $\text{rms_norm}(\text{activities}) < \sim 1e-3$ rather than $\text{rms_norm}(dz/dt)$, so it never fires and inference simply runs the clock out.

The fix, and why it is safe

Multiply the inference vector field by N , cancelling the $1/N$, so $\max_t1$ is the per-sample time. The weight step keeps the $1/N$, so gradients remain a batch mean and learning rates carry over unchanged. Verified two ways: the settled activities agree with jpc integrated to $t1 = 40N$ at $\cos = 1.000000$ (max relative difference $\sim 6e-4$), confirming the fixed point is untouched; and at the real bench $N = 2048$ the residual goes $0.988 \rightarrow 0.0037$. Cost is not a factor — 0.199 s/update against 0.221–0.254 s before — because past equilibrium the adaptive solver grows its step.

7. The ablation this unblocked, and what it overturned

Bandit, favor_suboptimal init ($\pi_0 = 0.018$), 60k steps, seeds 1–5, both PC algorithms, arms paired by seed; 20/20 runs completed. The arms are residual 0.90 (unsettled) against 0.003 (settled).

Final performance shows nothing — but it cannot: both arms reach 5/5 success at $\pi \approx 0.98$, i.e. the task is saturated at this budget.

algorithm	unsettled	settled	paired Δ	verdict
pc_reinforce	0.989 ± 0.003	0.987 ± 0.003	-0.002 ± 0.002	n.s.
pc_actor_critic	0.992 ± 0.003	0.977 ± 0.008	-0.015 ± 0.007	n.s.

Steps-to-threshold uses the whole learning curve instead of its endpoint, and is unambiguous — settling is **slower**:

algorithm	paired Δ steps to $\pi \geq 0.9$	t(4)	95% CI
pc_reinforce	+3661 $\pm$ 407	+9.00	[+2532, +4790]
pc_actor_critic	+5085 $\pm$ 1702	+2.99	[+361, +9809]

All five pc_reinforce seeds slower; four of five pc_actor_critic slower with one tie. t_crit at $n = 5$ is 2.776, so both clear it. (A 2·SEM rule would have mislabelled the final- π result as significant; these use a paired t-test.)

This falsifies the §5b prediction on this task — but does not overturn §5b. Measured directly in the bandit's geometry (discrete softmax, $1 \rightarrow 32 \rightarrow 2$), settled against unsettled gives $\cos = 0.67\text{--}0.89$ (mean ≈ 0.78) and a norm ratio of $1.05\text{--}1.65$: inference rotates the update substantially here, unlike the Gaussian $17 \rightarrow 64 \rightarrow 12$ geometry where $\cos(d_{BP}, d_{EQ}) \approx 0.99$. And the slowdown is not a step-size artifact, since settled steps are larger yet converge later. The lesson is that **“inference is inert” is specific to the geometry it was measured on**. The MuJoCo/Gaussian case has not been run — it needs a GPU — and is the open question.

8. What is established, and what is not

Established. Alignment with the natural gradient falls monotonically with inference time in all four regimes; the final-layer share stays ≥ 0.92 even at equilibrium, so the update is a final-layer delta rule; the only natural-gradient ingredient present is the target's F_{out}^{-1} ; the trust-region rescaling S is near-isotropic and directionally inert in this architecture, including in the linear case where the result is exact; the `log_std` clamp truncates $\sim 81\%$ of targets at `bench_target_scale`; and production inference ran at $\sim 1\%$ settled, now fixed and verified.

Not established. That PC has no trust-region property — the instrument used here cannot test the saddle-escape claim, and goes inert precisely at a saddle. That settling does or does not matter on MuJoCo — it changed the bandit materially, and the geometry differs. That any of this explains why PC succeeds on the bandit (0.968 ± 0.029 , 9/10, where PPO and REINFORCE reach ~ 0.02 , 0/10) while failing on continuous control: settling more makes the bandit slower, so the advantage is not “more inference”.

Limitations to state alongside any of these numbers

Sections 2–5 are initialisation geometry: weights at init plus σ -head bias shifts, a synthetic though distributionally matched batch, single seed, $n = 256$, one architecture family. Trained checkpoints may sit in better-conditioned geometry where S is anisotropic — that is the single most likely way the inertness result softens, and the real-checkpoint probe that would settle it is specified but unbuilt. The depth sweep varies architecture but stays at init. $\cos(d_{EQ}, d_{NG})$ uses a best-fit damping λ per cell. Section 7 is the only real-training result and is bandit-only, at $n = 5$.

Reproducing

All CPU, no environment needed:

```
python scripts/probe_natural_gradient.py
python scripts/probe_trust_region_metric.py
python scripts/probe_trust_region_metric.py --act-fn linear
python scripts/probe_inference_settling.py
python scripts/run_settled_ablation.py --seeds 1 2 3 4 5
```

Logs in `results/ng_probe_rerun/` and `results/settled_ablation/`. Narrative versions of these findings are in `docs/PCPG_GEOMETRY_FINDINGS.md` and `docs/PCPG_NATURAL_GRADIENT_PROBES.md`.